

Lab07 Spring Database

นายอนันต์เอกก์ ใหญ่พงศกร 673380430-9 sec3

ส่วนที่ 1: Software Design & Principles Explanation

1.1 สถาปัตยกรรมและ GRASP Patterns

ในการออกแบบระบบ Game Catalog ได้มีการนำหลักการ GRASP (General Responsibility Assignment Software Patterns) มาประยุกต์ใช้ในการแบ่งหน้าที่ความรับผิดชอบของแต่ละคลาสอย่างเหมาะสม ดังนี้:

- Controller Pattern (GameController): ทำหน้าที่เป็นจุดรับ HTTP Request จาก Web Browser ควบคุม Navigation และเป็นตัวกลางในการรับ-ส่งข้อมูลระหว่าง Presentation Layer (Thymeleaf UI) กับ Business Logic
- Information Expert:
 - o Game (Entity): เป็นผู้เชี่ยวชาญด้านข้อมูลคุณลักษณะของเกม (Title, Price, DiscountType ฯลฯ)
 - o DiscountStrategy: แต่ละ Concrete Class เป็นผู้เชี่ยวชาญการคำนวณส่วนลดเฉพาะประเภท
 - o GameRepository: เป็นผู้เชี่ยวชาญการเข้าถึงและสอบถามข้อมูล (Query) ในฐานข้อมูล PostgreSQL
- Low Coupling & High Cohesion: การแยกโค้ดออกเป็น Layered Architecture ช่วยให้แต่ละคลาสมีหน้าที่เฉพาะเจาะจงชัดเจน (High Cohesion) และแต่ละ Layer ไม่ยึดติดกันอย่างแน่นหนา (Low Coupling) เมื่อเกิดการเปลี่ยนแปลงในนิยามฐานข้อมูลหรือกฎการคำนวณส่วนลด จะไม่ส่งผลกระทบต่อส่วนแสดงผล UI
- Indirection: GameService ทำหน้าที่เป็นชั้นคั่นกลาง (Indirection) ระหว่าง GameController กับ GameRepository และ DiscountContext ช่วยลดการพึ่งพาโดยตรงระหว่าง Presentation Layer กับ Data Access Layer

1.2 High-Level SOLID Principles

ระบบได้รับการออกแบบตามหลักการ SOLID Principles เพื่อให้โค้ดมีความยืดหยุ่น ขยายง่าย และง่ายต่อการบำรุงรักษา:

- S -> Single Responsibility Principle (SRP): ทุกคลาสมีหน้าที่รับผิดชอบเพียงอย่างเดียว
 - o GameController รับผิดชอบจัดการ HTTP Requests และ Render หน้าเว็บ
 - o GameService รับผิดชอบ Business Logic และการเรียงลำดับขั้นตอนทำงาน
 - o GameRepository รับผิดชอบการ CRUD ข้อมูลลงฐานข้อมูล
 - o DiscountStrategy รับผิดชอบคำนวณส่วนลดราคาเกม

- O -> Open/Closed Principle (OCP): ระบบเปิดให้ขยายการทำงาน แต่ปิดการแก้ไขโค้ดเดิม หากต้องการเพิ่มโปรโมชั่นใหม่ (เช่น ส่วนลด VIP 30%) สามารถสร้างคลาสใหม่ที่ implement DiscountStrategy ได้ทันที โดยไม่ต้องแก้ไขโค้ดเดิมใน GameService หรือ DiscountContext
- L -> Liskov Substitution Principle (LSP): ทุกคลาสที่ implement DiscountStrategy (NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy) สามารถนำไปใช้งานแทนที่ DiscountStrategy ใน DiscountContext ได้อย่างสมบูรณ์โดยไม่ทำให้ระบบทำงานผิดพลาด
- I -> Interface Segregation Principle (ISP): Interface DiscountStrategy มีการกำหนดเฉพาะ Method ที่จำเป็นสำหรับการคำนวณส่วนลด ไม่มีการบังคับให้คลาสลูกต้อง implement method ที่ไม่ได้ใช้งาน
- D -> Dependency Inversion Principle (DIP): คลาสระดับสูง (GameController, GameService) ไม่ได้พึ่งพากลยักระดับล่างโดยตรง แต่พึ่งพา Abstraction ผ่าน Interface (GameRepository, DiscountStrategy) และใช้ Constructor Injection ในการฉีด Dependency เข้ามาใช้งาน

1.3 Strategy Pattern (การคำนวณส่วนลดราคาเกม)

เพื่อรองรับกลยุทธ์ส่วนลดราคาเกมที่หลากหลายและเปลี่ยนแปลงได้ตามโปรโมชั่น จึงนำ Strategy Pattern มาประยุกต์ใช้ดังนี้:

- Component Structure:
 1. DiscountStrategy (Interface): กำหนดสัญญาบริการในการคำนวณราคาสุทธิ
 2. NoDiscountStrategy (Concrete Strategy): คำนวณราคาปกติ (ส่วนลด 0%)
 3. StudentDiscountStrategy (Concrete Strategy): คำนวณราคาส่วนลดนักศึกษา (ลด 10%)
 4. SeasonalSaleStrategy (Concrete Strategy): คำนวณราคาส่วนลดเทศกาล (ลด 20%)
 5. DiscountContext (Context Class): ทำหน้าที่เลือกและเรียกใช้ Strategy ที่เหมาะสมตามประเภทส่วนลด (discountType)
- ประโยชน์ต่อ Open/Closed Principle (OCP): ช่วยแยกอัลกอริทึมการคำนวณราคาออกจากคลาสจัดการข้อมูล ทำให้สามารถเพิ่มกลยุทธ์ส่วนลดแบบใหม่ๆ เข้าสู่ระบบในอนาคตได้ง่ายดาย เพียงแค่เพิ่มคลาส Strategy ใหม่โดยไม่ต้องแก้ไขโค้ดการคำนวณเดิม

1.4 Layered Architecture & Execution Flow

ระบบแบ่งออกเป็น 4 Layer หลัก ได้แก่ Presentation Layer -> Service Layer -> Data Access Layer -> Database Layer

ลำดับการทำงาน (Execution Flow) เมื่อมีการเรียกใช้งานเว็บ:

1. HTTP Request: ผู้ใช้ส่งคำร้องขอผ่าน Browser (เช่น กดเพิ่มเกมใหม่ หรือเปิดหน้า /games)
2. Controller Layer: GameController ตรวจสอบ HTTP Request และดึงข้อมูลจาก Form Binding Object (Game)
3. Service Layer: GameController ส่งต่อวัตถุ Game ไปยัง GameService
4. Strategy Calculation: GameService เรียกใช้งาน DiscountContext เพื่อคำนวณราคาสุทธิ (finalPrice) ตาม Strategy ส่วนลดที่เลือก
5. Database Operation: GameService สั่งการ GameRepository (Spring Data JPA) เพื่อทำการบันทึกหรือค้นหาข้อมูลใน PostgreSQL Database

6. Response Rendering: GameController นำผลลัพธ์ฝากไว้ใน Model และสั่งให้ Thymeleaf Engine ประมวลผลสร้าง HTML Page ส่งกลับไปแสดงผลบน Browser ของผู้ใช้

ส่วนที่ 2: Code Implementation & Explanation

2.1 โครงสร้างซอร์สโค้ด (Package Structure)

```
None
src/main/java/com/example/lab7/
├── Lab7Application.java           # Main Application Entry Point
├── model/
│   └── Game.java                 # Entity Class mapped to PostgreSQL 'games'
table
├── repository/
│   └── GameRepository.java       # Spring Data JPA Repository Interface
├── strategy/
│   ├── DiscountStrategy.java    # Strategy Interface
│   ├── NoDiscountStrategy.java  # Concrete Strategy (0%)
│   ├── StudentDiscountStrategy.java # Concrete Strategy (10%)
│   ├── SeasonalSaleStrategy.java # Concrete Strategy (20%)
│   └── DiscountContext.java     # Strategy Context & Factory
├── service/
│   └── GameService.java         # Business Logic & Service Layer
└── controller/
    └── GameController.java     # Spring MVC Controller Layer
```

2.2 การประยุกต์ใช้ Dependency Injection (Constructor Injection)

โปรเจกต์นี้เน้นการใช้ **Constructor Injection** ในการจัดการ Dependency เพื่อความปลอดภัย และรองรับการทำ Unit Test ได้ง่าย:

- **Controller Layer Injection:** GameController รับ GameService ผ่าน Constructor

Java

```
@Controller
@RequestMapping("/games")
public class GameController {
    private final GameService gameService;

    // Constructor Injection
    public GameController(GameService gameService) {
        this.gameService = gameService;
    }
    ...
}
```

- **Service Layer Injection:** GameService รับ GameRepository และ DiscountContext ผ่าน Constructor

Java

```
@Service
public class GameService {
    private final GameRepository gameRepository;
    private final DiscountContext discountContext;

    // Constructor Injection
    public GameService(GameRepository gameRepository, DiscountContext discountContext) {
        this.gameRepository = gameRepository;
        this.discountContext = discountContext;
    }
    ...
}
```

ส่วนที่ 3: Web Application & Database Screenshots

3.1 หน้าจอการเพิ่มเกมใหม่ (Create)

Game Catalog CP353002 — Lab 7

← กลับไปรายการเกม

+ เพิ่มเกมใหม่

กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

↓ ชื่อเกม

Valorant - 673380430-9 sec

🎮 แนวเกม (Genre) 🖥️ แพลตฟอร์ม (Platform)

FPS, Shooting PC

⭐ คะแนน (Rating) 📅 วันวางจำหน่าย (Release Date)

8 08/13/2026

💰 ราคาปกติ (บาท)

600

% ส่วนลด (Strategy)

- ส่วนลดนักศึกษา (10%)
- ราคาปกติ (0%)
- ส่วนลดนักศึกษา (10%)
- ส่วนลดเทศกาล (20%)

💾 บันทึก 🔄 ยกเลิก

รูปที่ 3.1: หน้าฟอร์มเพิ่มเกมใหม่ (ระบุรหัสนักศึกษา 663380430-9 Section 3 และเลือกส่วนลดนักศึกษา 10%และ20%)

3.2 หน้าจอแสดงรายการเกมทั้งหมด

Game Catalog CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

📁 รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

➕ เพิ่มเกมใหม่

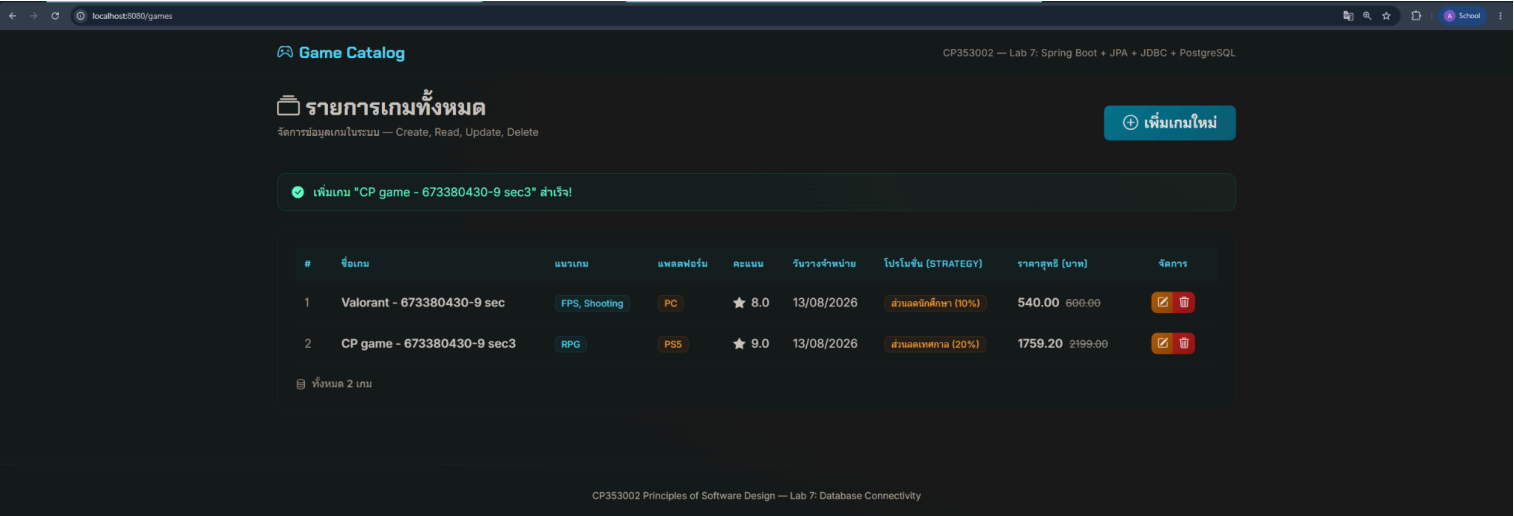
✅ เพิ่มเกม "Valorant - 673380430-9 sec" สำเร็จ!

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Valorant - 673380430-9 sec	FPS, Shooting	PC	★ 8.0	13/08/2026	ส่วนลดนักศึกษา (10%)	540.00 600.00	✎ 🗑

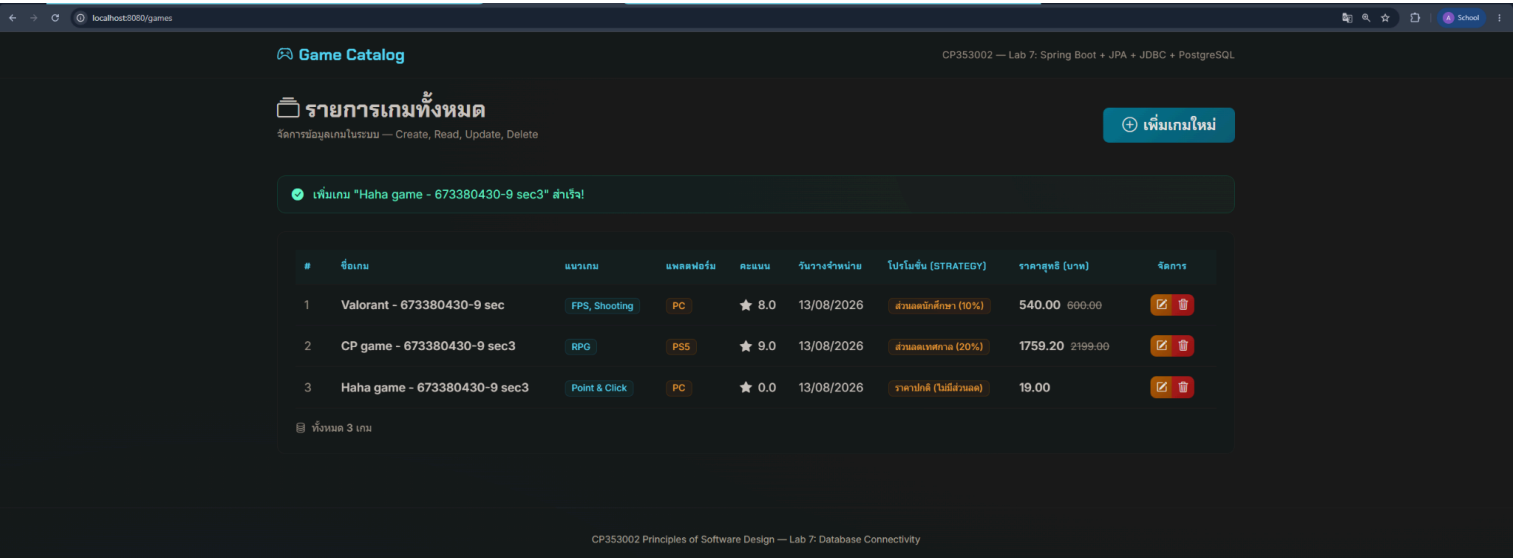
📄 ทั้งหมด 1 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

รูปที่ 3.2.1: หน้าแสดงรายการเกมทั้งหมด พร้อมแสดงราคาสุทธิที่คำนวณผ่าน Strategy Pattern



รูปที่ 3.2.2: หน้าแสดงรายการเกมทั้งหมด พร้อมแสดงราคาสุทธิที่คำนวณผ่าน Strategy Pattern



รูปที่ 3.2.3: หน้าแสดงรายการเกมทั้งหมด พร้อมแสดงราคาสุทธิที่คำนวณผ่าน Strategy Pattern

3.3 หน้าจอแก้ไขข้อมูลเกม

Game Catalog CP353002 — Lab 7

← กลับไปรายการเกม

แก้ไขข้อมูลเกม

แก้ไขข้อมูลเกม: Haha game - 673380430-9 sec3

ชื่อเกม

Haha game - 673380430-9 sec3

แนวเกม (Genre) Platform

Point & Click PC

คะแนน (Rating) ราคาปกติ (บาท) % ส่วนลด (Strategy)

10 19.0 ราคาปกติ (0%)

วันวางจำหน่าย (Release Date)

08/13/2026

บันทึก ยกเลิก

รูปที่ 3.3: หน้าฟอร์มแก้ไขข้อมูลเกม

Game Catalog CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

เพิ่มเกมใหม่

แก้ไขข้อมูลเกมสำเร็จ!

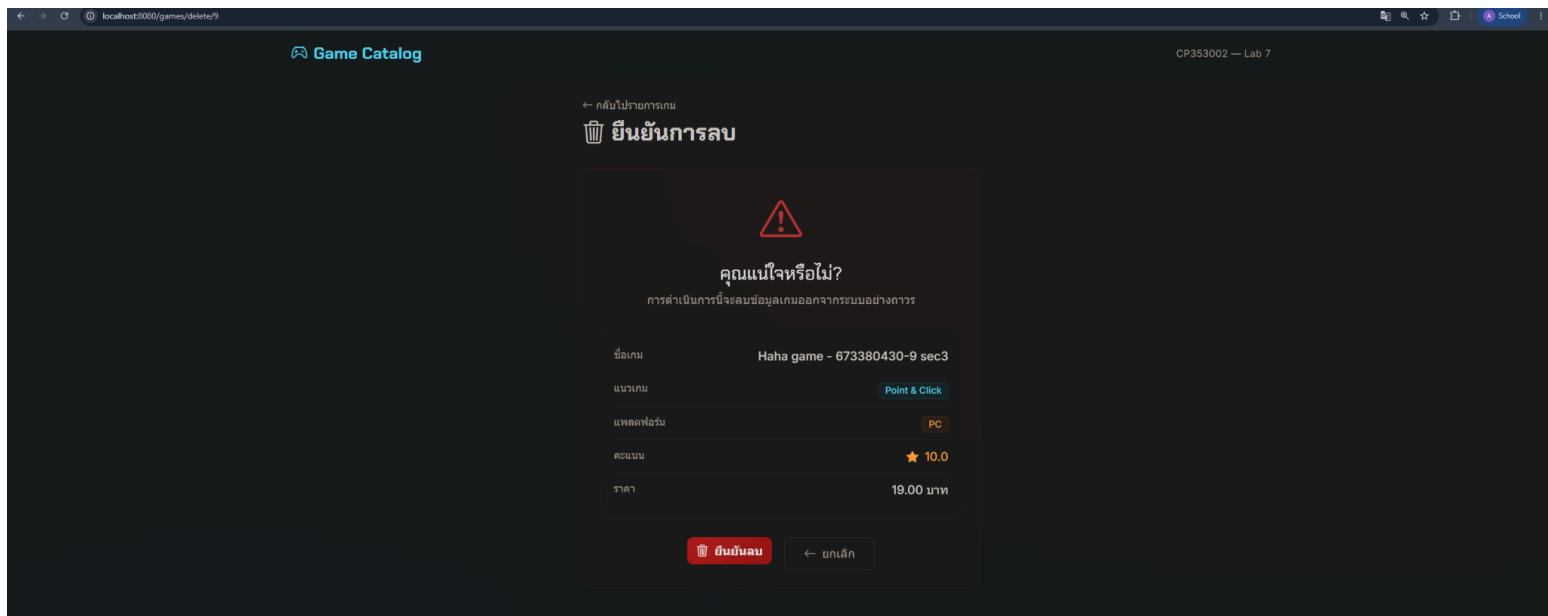
#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Valorant - 673380430-9 sec	FPS, Shooting	PC	★ 8.0	13/08/2026	ส่วนลดนักศึกษา (10%)	540.00 600.00	แก้ไข ลบ
2	CP game - 673380430-9 sec3	RPG	PS5	★ 9.0	13/08/2026	ส่วนลดพนักงาน (20%)	1759.20 2199.00	แก้ไข ลบ
3	Haha game - 673380430-9 sec3	Point & Click	PC	★ 10.0	13/08/2026	ราคาปกติ (ไม่มีส่วนลด)	19.00	แก้ไข ลบ

ทั้งหมด 3 เกม

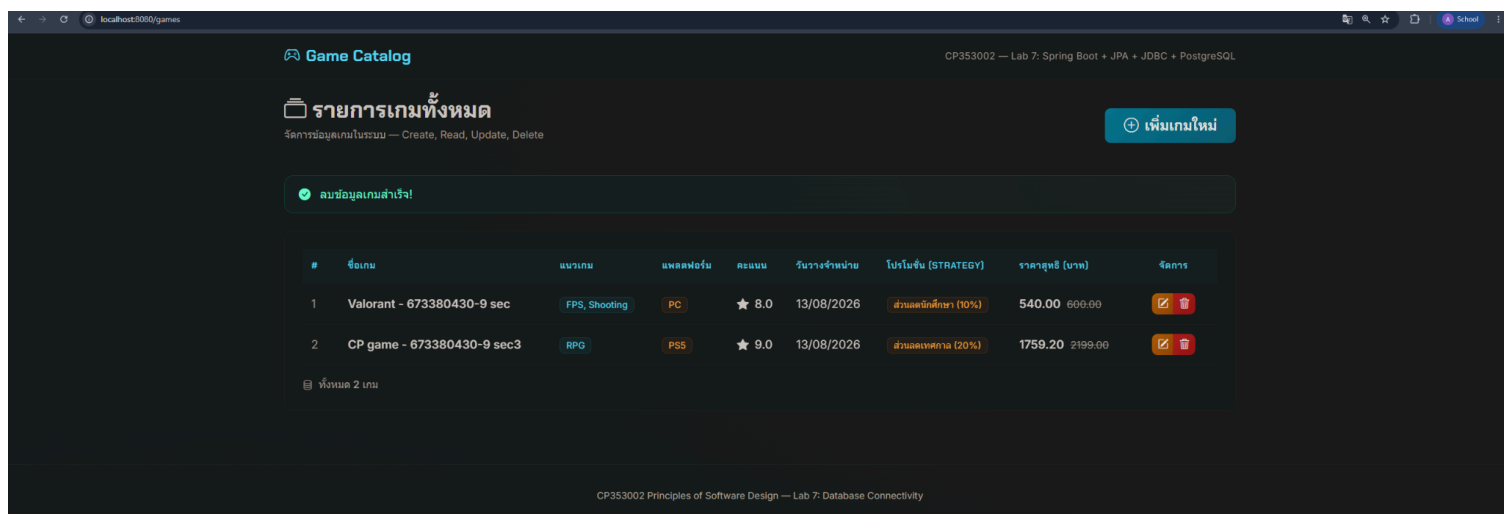
CP353002 Principles of Software Design — Lab 7: Database Connectivity

รูปที่ 3.3.4: หน้าจอยืนยันการแก้ไขเกม และผลลัพธ์หลังการแก้ไข

3.4 หน้าจอยืนยันและการลบข้อมูลเกม



รูปที่ 3.4.1: หน้าจอยืนยันการลบเกม



รูปที่ 3.4.2: หน้าจอผลลัพธ์หลังทำการลบ

3.5 หน้าจอตรวจสอบข้อมูลใน PostgreSQL Database

```
lab7demo=# SELECT * FROM games;
 id | discount_type | genre      | platform | price | rating | release_date | title
-----+-----+-----+-----+-----+-----+-----+-----
  7 | STUDENT       | FPS, Shooting | PC       |  600 |      8 | 2026-08-13 | Valorant - 673380430-9 sec
  8 | SEASONAL     | RPG         | PS5      | 2199 |      9 | 2026-08-13 | CP game - 673380430-9 sec3
(2 rows)

lab7demo=#
```

รูปที่ 3.5: ข้อมูลตาราง games ที่ถูกจัดเก็บจริงในฐานข้อมูล PostgreSQL